

Work Sheet Application Whitelisting (In-Class Exercise)

1. Application Whitelisting

Learn about **Windows Defender Application Control** (WDAC) integrated in the Microsoft Windows operating system versions Enterprise and Pro. What is the goal of application whitelisting? This is what you have already done while working on the first exercise sheet:

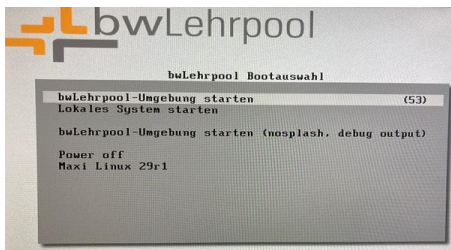
- 1.1. Explain how application whitelisting works with **WDAC (Windows Defender Application Control)**? How is whitelisting different from blacklisting?
- 1.2. Does WDAC restrict users/processes with respect to executing code? How?
- 1.3. Does WDAC restrict users/processes with respect to reading and writing files? How?
- 1.4. What are the different **file rules** that you can use with WDAC?
- 1.5. What is the purpose of the **audit mode** in WDAC?
- 1.6. How do you debug WDAC policies, i.e., how do you find out that they are effective and where are WDAC events logged?

How can applications be selected for a whitelist in WDAC? Look up WDAC documentation on <https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/> to find out more on rules based on file names, file paths, hash values, publishers, and Intelligent Security Graph.

2. Work environment

You may use your own instance of Windows 11 Pro/Enterprise on a laptop. **It is recommended to use a VM** that can be reset. Do not experiment on your production laptop outside a VM. You might accidentally configure a policy that prevents you from using your laptop and you do not want that to happen. **You find sample WDAC policies in Moodle.** You need to use an account that is a member of the Administrators group to set up and modify policies. You can use that same account for testing as WDAC policies apply per machine and also affect administrator accounts.

If you prefer not to use your own laptop, you can use one of the computers in the lab. On all machines a Windows 11 Enterprise instance is available.



Restart the computer and press F12 while booting. This brings up a menu called “bwLehrpool Bootauswahl”. Here you select “bwLehrpool-Umgebung starten”.



You need to log on using your HTWG account name and HTWG password. The next step is to select the OS instance that you want to start. That is “IT-Security”. After selection you click “Start”.



The VM is started and you are automatically logged on as user “Administrator” without password. The “Administrator” account is a member of the local administrators group.

You are now ready to work.

3. Hints

You can modify WDAC policies using Windows PowerShell. You may execute the PowerShell commands interactively. You do not need to put them in a script file first. You may enhance one of the sample policies and save the enhanced policy to a new file name. Once you have configured a policy and have stored it as an XML file, you need to convert it into a CIP file (see first task below). The CIP file needs to be placed in the folder `C:\Windows\System32\CodeIntegrity\CIPolicies\Active`. Then you refresh the WDAC policy. The tool to refresh the policy is called "Refresh WDAC Policy". You may download `RefreshPolicy(AMD64).exe` from <https://www.microsoft.com/en-us/download/details.aspx?id=102925>.

Do not restart the operating system – restarting a VM in bwLehrpool leads to the whole machine being reset and you will lose all your work, i.e., all the commands you typed and all the files you modified.

4. Tasks

1. *Set up and activate a **policy in audit mode** and check the logs how the policy works*

There is a sample policy to achieve that, e.g., `DenyAllAudit.xml`. Copy that XML file to a folder of your choice, launch PowerShell and navigate to your chosen folder.

PowerShell (three lines):

```
[xml]$PolicyXml = Get-Content .\DenyAllAudit.xml
$PolicyId = $PolicyXml.SiPolicy.PolicyId
ConvertFrom-CIPolicy .\DenyAllAudit.xml $PolicyId ".cip"
```

You can view the log entries with `eventvwr.exe` and open the log Applications and Services Logs → Microsoft → Windows → CodeIntegrity → Operational. Watch out for events with ID 3076.

2. *Set up and activate a **policy that restricts execution** and confirm that the policy works*

There is a sample policy to achieve that, e.g., `DefaultWindows_Enforced.xml`. Alternatively, you may use the policy from the first task as a baseline, and use `Set-CIPolicyIdInfo` and `Set-RuleOption` to remove option 3 (audit mode). Document test cases that demonstrate that allowed applications can be executed and others are blocked.

<https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/deployment/enforce-appcontrol-policies>

3. *Extend the **policy to include an exception for one application** and confirm that the policy works*

You may want to start with a sample policy and extend that. Choose inclusion rules based on file name, file path (simple), hash value, or publisher (more complicated). If you have time, create different policies based on different rules. Document test cases that demonstrate that allowed applications can be executed and others are blocked. You may want to test your policy in audit mode first before you enforce it. If path-based rules do not work at first, look up "Understand App Control for Business policy rules and file rules" on learn.microsoft.com.

If you struggle with PowerShell, try using the App Control Wizard: <https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/appcontrol-wizard>